

 Modules

^

[Module 1](#)

[Module 2](#)

[Module 3](#)

[Module 4](#)

[Module 5](#)

[Module 6](#)

[Module 7](#)

[Module 8](#)

[Module 9](#)

[Module 10](#)

[Module 11](#)

[Module 12](#)

[Module 13](#)

[Module 14](#)

[Module 15](#)



[Projects](#)



[Resource hub](#)

[Project brief](#)

[Your submission](#)

Brief

1. Project Summary

LearnTrack is a console-based Student & Course Management System built using **Core Java**.

It will allow admins to manage:

- Students
- Courses
- Enrollments

The goal is to practice:

- Java basics (variables, data types, control flow)
- Classes, objects, constructors
- Static vs instance members
- OOP principles (encapsulation, basic inheritance, polymorphism)
- Collections (ArrayList)
- Basic exception handling
- Clean, readable code and modular design

This project is intentionally kept focused on **fundamentals** without advanced topics like concurrency, streams, or heavy design patterns. Those can be added later as the cohort progresses.

2. Learning Objectives

By completing LearnTrack, mentees should be comfortable with:

1. **Java setup & compilation**
 - Using JDK, understanding JDK vs JRE vs JVM at a basic level
 - Compiling and running Java files via IDE and/or terminal

2. **Core Java Syntax & Basics**

- Packages, classes, main method
- Data types (primitives vs reference types)
- Variables and their scopes (local, instance, static)
- Typecasting basics

3. **OOP Fundamentals**

Ask me anything

- Creating classes and objects
- Constructors (default, parameterized, constructor overloading)
- Encapsulation using private fields + getters/setters
- Basic inheritance and method overriding
- Simple polymorphism examples

4. Logic & Control Flow

- if/else, switch, loops (for, while, do-while)
- Using conditions and loops in menu-driven console applications

5. Collections

- Array vs ArrayList (why ArrayList is preferred for dynamic data)
- Storing and retrieving objects using ArrayList

6. Exception Handling (Basic)

- try-catch blocks
- Handling invalid input, e.g., wrong menu option, empty lists

7. Clean Code Mindset

- Small functions
- Clear method names
- Separation of concerns (entity vs service vs UI)

3. Project Requirements & Suggested Marks Breakdown (Total 100)

A. Environment Setup & JVM Understanding (10 marks)

- Install and configure Java (JDK).
- Create a file: docs/Setup_Instructions.md with:
 - JDK version used
 - Screenshots or brief explanation of "Hello World" program run.
- Create docs/JVM_Basics.md explaining in simple language:
 - What is JDK, JRE, JVM
 - What is bytecode
 - What does "write once, run anywhere" mean (1–2 short paragraphs)

B. Package Structure & Basics (10 marks)

Base package (example):
com.airtribe.learntrack

Recommended sub-packages:

- entity – Student, Course, Enrollment
- service – StudentService, CourseService, EnrollmentService

- ui – Menu / Console UI ([Main.java](#))
- exception – Custom exceptions (e.g., EntityNotFoundException, InvalidInputException)
- util – Simple helper classes (e.g., IdGenerator, InputValidator)
- docs – Documentation files

You must demonstrate:

- Proper use of **packages**
- public, private, and default access where appropriate
- Use of **static** variables/methods where it makes sense (e.g., ID counters)

C. Core OOP Implementation (40 marks)

1. Entities & Encapsulation (15 marks)

Create these core entity classes:

- Student
 - Fields: id, firstName, lastName, email, batch, active (boolean)
- Course
 - Fields: id, courseName, description, durationInWeeks, active
- Enrollment
 - Fields: id, studentId, courseId, enrollmentDate, status (e.g., "ACTIVE", "COMPLETED", "CANCELLED" as String or simple enum if you want slightly advanced)

Requirements:

- Use **private fields + public getters/setters** (Encapsulation)
- Add **parameterized constructors** (and default constructor if needed)
- Demonstrate **constructor overloading** in at least one class (e.g., Student with and without email)

2. Inheritance & Basic Polymorphism (10 marks)

Introduce a small hierarchy to practice:

- Person (base class)
 - Fields: id, firstName, lastName, email
- Student extends Person
- Optionally, introduce Trainer extends Person (even if not used much) to show inheritance

Show:

- Use of super in constructors
- Simple method overriding, e.g., getDisplayName() in Person and specialized behavior in Student or Trainer

3. Static, Methods, and Utility Classes (15 marks)

- Use a IdGenerator utility:
 - Static field like private static int studentIdCounter
 - Static methods like getNextStudentId(), getNextCourseId()

Ask me anything

- Create meaningful methods:
 - In services: addStudent, removeStudent, updateStudent, listStudents
 - Use methods with different parameters (overloading example).
-

D. Application Logic & Menu-Driven Console UI (25 marks)

The core of the project is a **menu-based console application** in [Main.java](#):

Example features:

1. Student Management

- Add new student
- View all students
- Search student by ID
- Deactivate a student (set active = false instead of deleting)

2. Course Management

- Add new course
- View all courses
- Activate/Deactivate a course

3. Enrollment Management

- Enroll a student in a course
- View enrollments for a student
- Mark enrollment as completed/cancelled

Implementation expectations:

- Use **ArrayList** to store Students, Courses, and Enrollments.
- All data can be **in-memory only** (no file DB needed for now).
- Use **loops and conditionals** to show and process menu options.
- Handle invalid input gracefully:
 - Option not found
 - Non-existent student/course IDs
- Keep logic in **service classes**, and keep Main focused on:
 - Displaying menu
 - Reading input
 - Calling service methods

E. Basic Exception Handling (10 marks)

- Create at least one **custom exception**, for example:
 - EntityNotFoundException when a Student or Course is not found.

Ask me anything

- Use try-catch around input parsing (e.g., converting String to int) and menu operations.
- Show a clean user message instead of crashing the program.

F. Documentation & Clean Code (5 marks)

- [README.md](#) with:
 - Project description
 - How to compile and run
- docs/Design_Notes.md answering:
 - Why you used ArrayList instead of array
 - Where you used static members and why
 - Where you used inheritance and what you gained from it
- Follow simple **Clean Code** ideas:
 - Methods not too long
 - Meaningful names (addStudent, findCourseById, not fun1 or doWork)

 Untitled document.pdf 

Submission guidelines

1. Write a clear and concise README file.
2. A class diagram showing the relationships between your classes (Add this in your Readme)
3. To submit your project, please provide a link to your GitHub repository and your submission should be a PR.
4. **[Important] Make sure the repository is public.**

Deliverables

 GitHub PR link

 GitHub link



Hey there, I'm your guide for this project!

Ask me anything regarding this brief, I'd be happy to help you out.

Identify project components

Describe Java usage

List OOP principles

Explain exception handling

Ask me anything

Ask doubts or share resources

Add comment

Share tips, resources or anything else related to the assignment here

B *I* u    

Post

 The comments can be seen by everyone in the cohort

Comments: (0)

No comments yet. Be the first one to start a discussion.

Ask me anything